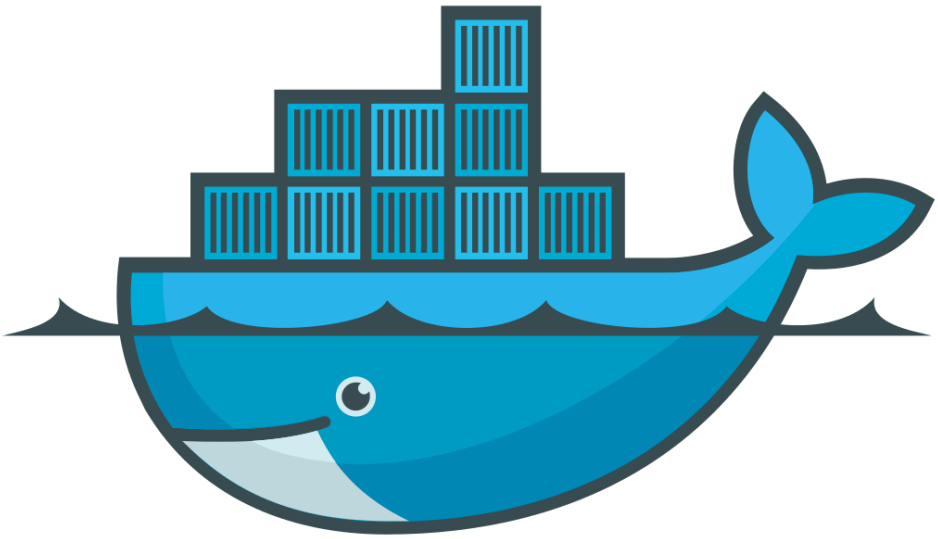
Inside the Docker Kernel | Process, Namespace & cgroup Isolation



Date: November 07, 2025

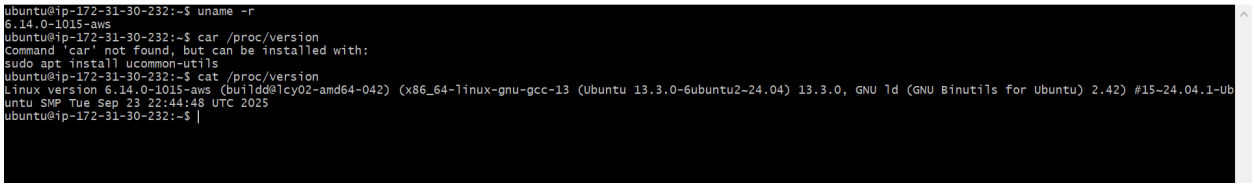
Danutha Imbulpitiya

1. Verifying Kernel Sharing

Approach

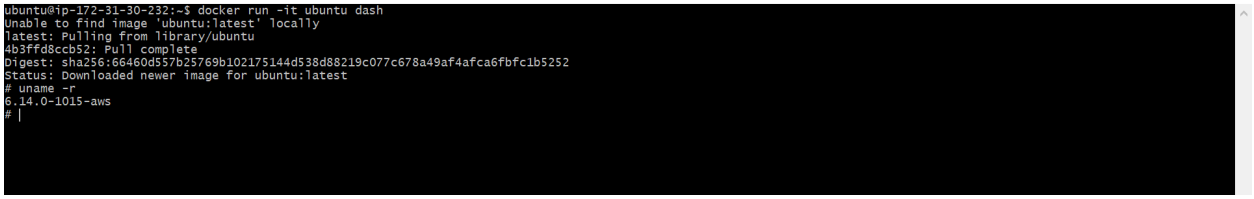
To prove that Docker containers share the host's kernel and do not run their own

1. Capture host kernel information

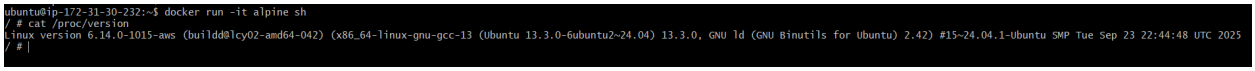


2. Launch containers with different base images and execute the same kernel queries inside

   Ubuntu image



   Alpine image



To prove Docker containers share the host kernel, I checked the kernel version on the host using uname -r and cat /proc/version. Then, I started two containers one with Ubuntu and one with Alpin and ran the same commands inside each. All three showed the exact same kernel version and build details, even though Ubuntu and Alpine are completely different Linux distributions. This proves containers don't run their own kernel they all use the host's kernel. Unlike virtual machines, which need a full guest OS and kernel, Docker containers are just isolated processes sharing the same kernel.

Proof That Containers Are Not Virtual Machines

Containers are not virtual machines because they share the host's kernel instead of running their own. VMs emulate full hardware, boot a complete guest OS, and include a separate kernel causing high overhead and slow startup. Containers, however, are just isolated Linux processes

using the same kernel as the host. They rely on namespaces and cgroups, not hardware virtualization.

2. Process and PID Mapping

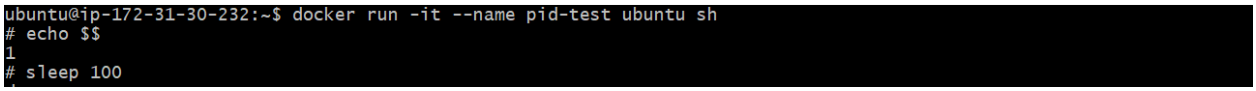
Approach

The overall approach demonstrates that container processes are just regular host processes by launching a container, identifying its internal PID, mapping it to the actual host PID via Docker tools, and tracing the process tree to show its place in the host's hierarchy under Docker's runtime components like containerd-shim.

Approach

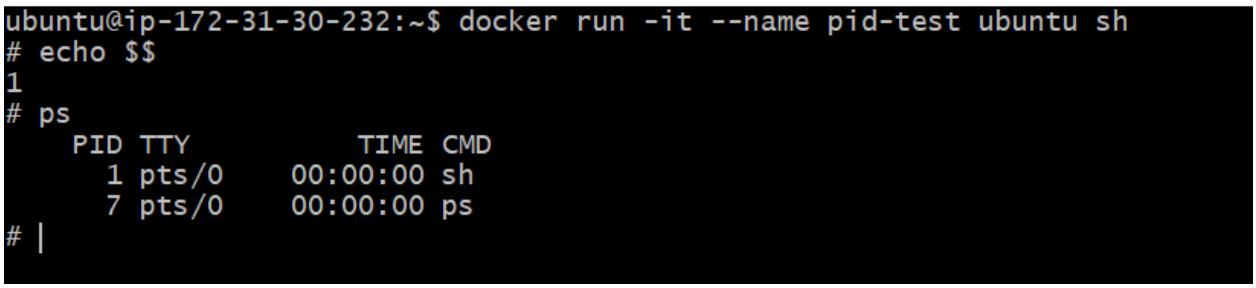
1. Launch the Container and Identify Container's PID 1

   Started an interactive Ubuntu container with a specific name and executed **sleep 100** inside the container to prevent immediate exit and allow observation on the host.



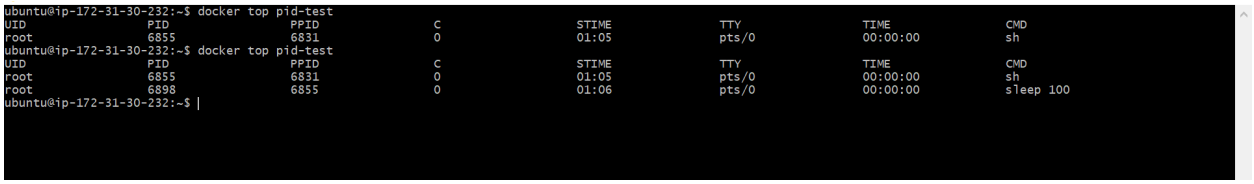
2. PID for the container

   Inside the container, ran **echo $$** to display the PID of the main shell this showed **1**, confirming it acts as the container's init process.



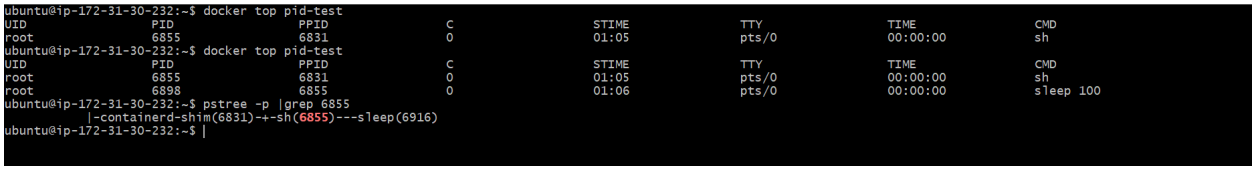
3. Host PID for the container process

   On the host, used docker top pid-test to list running processes inside the container and identify the host-level PID corresponding to container PID



4. Trace Process Hierarchy

   Ran pstree -p | grep <host_pid> to visualize the full process tree. This revealed the container process as a child of **containerd-shim**, proving it's a regular Linux process under Docker's runtime stack.



Process hierarchy

systemd (PID 1)
└── dockerd
└── containerd
└── containerd-shim (PID 6831)
└── sh(PID 6835) <-- Container's PID 1 inside
└── sleep(PID 6916)

Conclusion

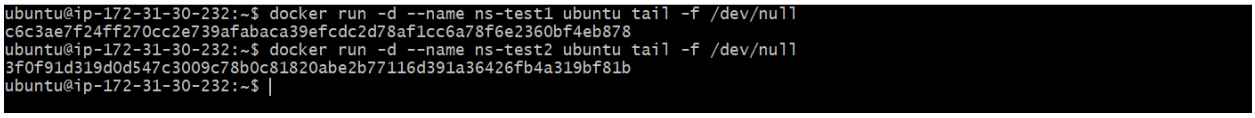
I launched an Ubuntu container and confirmed its main process shows PID 1 inside using echo $$. On the host, docker top revealed this maps to a regular high-numbered PID (6855), and **pstree** showed it as a child of **containerd-shim.** This proves **container processes are ordinary host processes** isolated only by **PID namespaces**, not running in a separate OS.

3. Exploring Namespace Isolation

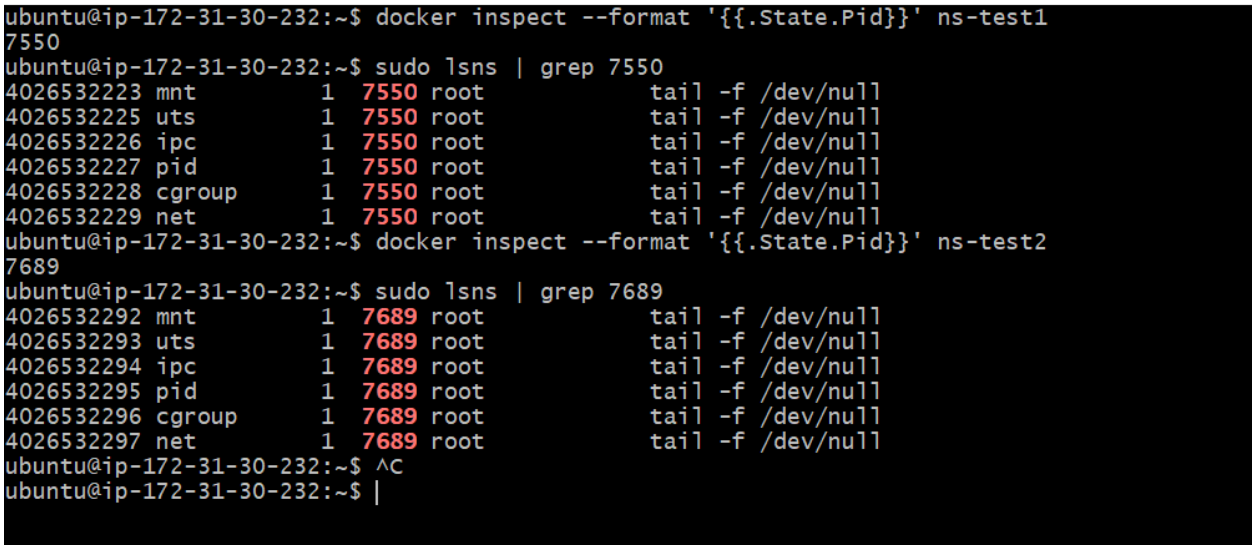
Approach

1. Launched two containers

   Used **tail -f /dev/null** to keep each container running indefinitely without consuming real resources, and the **-d (detached)** flag to run them in the background so I could inspect them from the host without staying attached. Launching two containers was essential to compare namespace isolation only by checking both could I confirm which namespaces are unique to each container and which are shared.



2. Used **docker inspect --format '{{.State.Pid}}'** to get their host PIDs (7550 and 7689) and ran sudo **lsns | grep** for each to list namespace types and inode numbers



   Docker runs containers as regular host processes, but inside the container, the PID appears as **1** due to namespace isolation. Using **docker inspect --format '{{.State.Pid}}'**, I extracted the **real Linux PID** on the host (e.g., 7550 or 7689) this is critical because **kernel namespace information** lives under **/proc/<pid>/ns/** and tools like **lsns** require the **host PID** to correctly identify and compare namespace inodes.

   lsns lists all active Linux namespaces on the host along with their unique **inode numbers** the key to determining isolation. Two processes belong to the **same namespace** if their inode numbers match, different inodes mean **separate, isolated**

namespaces. And **grep <PID>** filtered the output to show only the namespaces owned by the container's host PID, making it easy to compare isolation across containers.

Conclusion

All namespaces including **PID, NET, MNT, UTS, IPC and cgroup** had unique inodes for each container, proving complete namespace isolation. This shows Docker creates fully independent instances of every namespace per container, ensuring strong isolation across processes, networking, filesystems, hostnames, IPC.

4. Observing New Namespace Creation

Approach

1. Ran a lightweight container

```
ubuntu@ip-172-31-30-232:~$ docker run --rm alpine echo "test"

test
ubuntu@ip-172-31-30-232:~$ |
```

2. Attached strace to Docker daemon

Used sudo strace -f -e trace=clone,unshare -p $(pidof dockerd) to monitor Docker's system calls in real time. The **-f** flag followed child processes like **containerd and runc**, where **clone()** happens. Filtering with **-e trace=clone,unshare** focused only on namespace creation calls. Output was saved using 2>&1 | tee /tmp/docker-trace.log for analysis.

```
ubuntu@ip-172-31-30-232:~$ sudo strace -f -e trace=clone,unshare -p $(pidof dockerd) 2>&1 | tee /tmp/docker-trace.log
strace: Process 2373 attached with 12 threads
[pid  2698] --- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2373, si_uid=0} ---
[pid  2698] --- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2373, si_uid=0} ---
[pid  2375] --- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2373, si_uid=0} ---
[pid  2700] clone(child_stack=NULL, flags=CLONE_VM|CLONE_PIDFD|CLONE_VFORK|SIGCHLDstrace: Process 11811 attached
, parent_tid=[39]) = 11811
[pid  2700] --- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2373, si_uid=0} ---
[pid 11811] +++ exited with 1 +++
[pid  3104] --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=11811, si_uid=0, si_status=1, si_utime=0, si_stime=0} ---
[pid  3104] clone(child_stack=NULL, flags=CLONE_VM|CLONE_PIDFD|CLONE_VFORK|SIGCHLDstrace: Process 11813 attached
, parent_tid=[39]) = 11813
[pid 11813] +++ exited with 0 +++
[pid  3104] --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=11813, si_uid=0, si_status=0, si_utime=0, si_stime=0} ---
[pid  2700] --- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2373, si_uid=0} ---
[pid  2380] --- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2373, si_uid=0} ---
[pid  3104] --- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2373, si_uid=0} ---
[pid  3104] clone(child_stack=NULL, flags=CLONE_VM|CLONE_PIDFD|CLONE_VFORK|SIGCHLDstrace: Process 11856 attached
, parent_tid=[31]) = 11856
[pid 11856] +++ exited with 0 +++
[pid  2700] --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=11856, si_uid=0, si_status=0, si_utime=0, si_stime=0} ---
[pid  2700] clone(child_stack=NULL, flags=CLONE_VM|CLONE_PIDFD|CLONE_VFORK|SIGCHLDstrace: Process 11858 attached
, parent_tid=[31]) = 11858
[pid 11858] +++ exited with 0 +++
[pid  2700] --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=11858, si_uid=0, si_status=0, si_utime=0, si_stime=0} ---
[pid  3104] --- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2373, si_uid=0} ---
[pid  3104] --- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2373, si_uid=0} ---
[pid  2375] --- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2373, si_uid=0} ---
^C
ubuntu@ip-172-31-30-232:~$ |
```

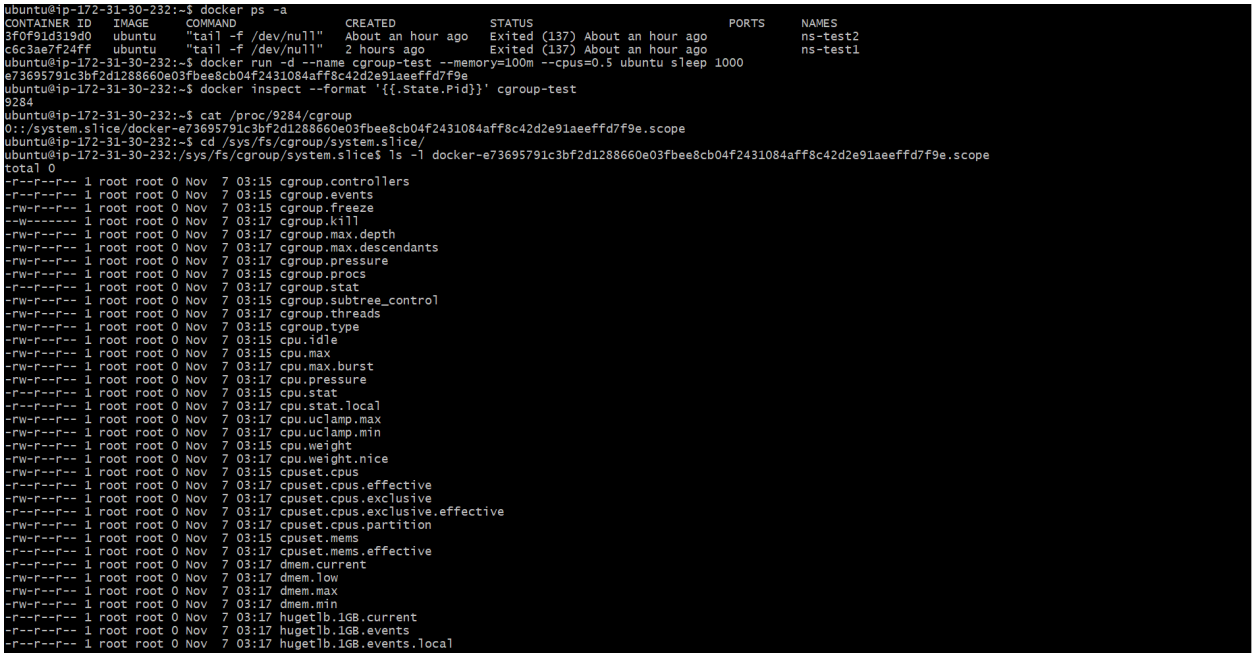
Conclusion

The strace capture showed Docker relying on clone() system calls with flags like C**LONE_VM, CLONE_VFORK, CLONE_PIDFD,** and **SIGCHLD** to create new namespaces during container startup no **unshare()** calls were observed. This confirms that namespace isolation is established directly at process creation via **clone()**, enabling efficient forking with immediate **PID** and memory separation. **CLONE_PIDFD** specifically isolates the **PID** namespace, while **CLONE_VM|CLONE_VFORK** optimizes resource sharing until **exec**. Overall, this proves Docker achieves lightweight isolation without post-fork modifications, using kernel primitives for secure, performant containers.

5. Investigating cgroup

Approach

1. **Launch the container with limits**



   Started a Ubuntu container named "cgroup-test" using docker run --name cgroup-test --memory=100m --cpus=0.5 ubuntu sleep 1000 to apply custom memory (100MiB) and CPU (0.5 cores) limits.
2. **Find the cgroup path**
   Used **docker inspect --format '{{.State.Pid}}' cgroup-test** to get the host PID (7584), then located the cgroup scope under **/sys/fs/cgroup/system.slice/docker-<container-id>.scope.**
3. **List active controllers**
   Used **ls /sys/fs/cgroup/system.slice/** to list all cgroup controllers (e.g., cpu, memory, io, pids) and their files.

4. **Check resource usage**

```
CONTAINER ID   NAME          CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
e73695791c3b   cgroup-test   0.00%     392KiB / 100MiB     0.38%     1.23kB / 126B   0B / 0B     1
```

Ran **docker stats** to monitor the container's CPU, memory, and other metrics, confirming the applied limits (e.g., MEM LIMIT as 100MiB).

5. **Verify quota files**

```
ubuntu@ip-172-31-30-232:/sys/fs/cgroup/system.slice$ cat /sys/fs/cgroup/system.slice/docker-e73695791c3bf2d1288660e03fbee8cb04f2431084aff8c42d2e91aeeffd7f9e.scope/cpu.max
50000 100000
ubuntu@ip-172-31-30-232:/sys/fs/cgroup/system.slice$ cat /sys/fs/cgroup/system.slice/docker-e73695791c3bf2d1288660e03fbee8cb04f2431084aff8c42d2e91aeeffd7f9e.scope/memory.ma
x
104857600
ubuntu@ip-172-31-30-232:/sys/fs/cgroup/system.slice$
ubuntu@ip-172-31-30-232:/sys/fs/cgroup/system.slice$
```

Ran **cat on files like /sys/fs/cgroup/.../cpu.max** and /**sys/fs/cgroup/.../memory.max**

6. **Observe throttling**: Noted low usage in docker stats (0.00% CPU, 0.38% MEM) since the container was idle, but limits were enforced as per quota files—no throttling seen yet due to no load.

```
CONTAINER ID   NAME          CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
e73695791c3b   cgroup-test   0.00%     392KiB / 100MiB     0.38%     1.23kB / 126B   0B / 0B     1
```

Conclusion

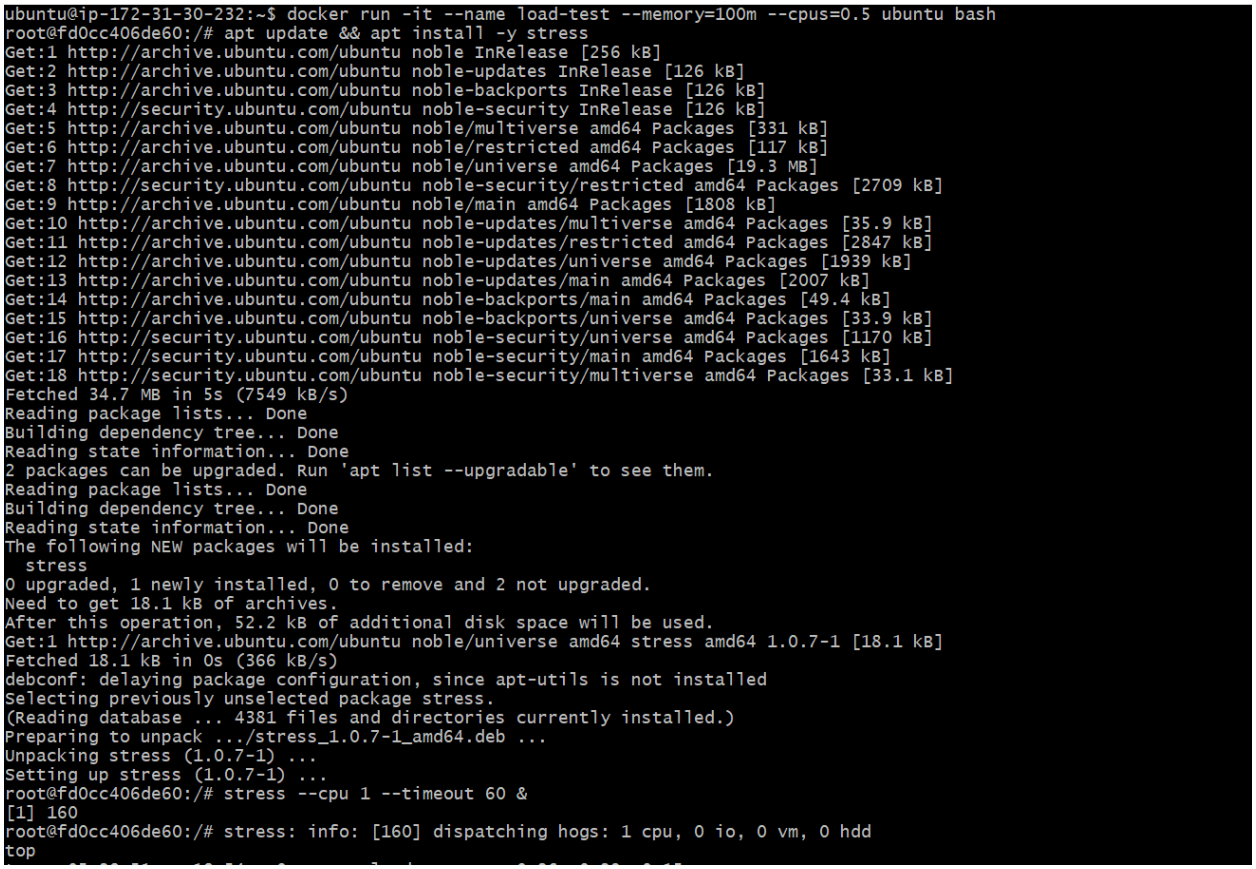
The cgroup path for the container was found under **/sys/fs/cgroup/system.slice/docker-<id>.scope**, with active controllers like cpu, memory, io, and pids visible. Quota files such as cpu.max (50% CPU share) and memory.max (100MiB) confirmed the custom limits were applied. In docker stats, the idle container stayed within bounds (0% CPU, ~392KiB memory used), showing no throttling, but this setup proves cgroups enforce resource isolation preventing overconsumption and enabling fair sharing on the host.

6. Resource Behavior Under Load

I started a container with tight limits using **docker run -it --name load-test --memory=100m --cpus=0.5 ubuntu bash**. Inside, installed **stress** with **apt update && apt install -y stress**. Then, I ran **stress --cpu 1 --timeout 60** to push CPU usage and **stress --vm 1 --vm-bytes 150M --timeout 60** to exceed memory. While the stress ran, I monitored real-time behavior using docker stats on the host and top inside the container to observe how cgroups enforced the limits.

Approach

1. Launched a container with strict limits



   Ran **docker run -it --name load-test --memory=100m --cpus=0.5 ubuntu bash** to cap memory at 100MiB and CPU at 0.5 cores.

   Inside the container, ran **apt update && apt install -y stress** to enable load testing.

   Executed **stress -- cpu 1 –timeout 60 &** to generate high CPU load for 60 seconds.

2. Monitored real-time metrics

Container stats

```
top - 05:27:42 up 18:53,  0 user,  load average: 0.67, 0.33, 0.15
Tasks:   4 total,   2 running,   2 sleeping,   0 stopped,   0 zombie
%Cpu(s): 25.0 us,  0.2 sy,  0.0 ni, 74.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :    914.2 total,    144.4 free,    454.7 used,    486.4 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.    459.6 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    161 root      20   0    3620    456    456 R  49.2   0.0   0:25.54 stress
      1 root      20   0    4588   3716   3332 S   0.0   0.4   0:00.03 bash
    160 root      20   0    3620   1784   1784 S   0.0   0.2   0:00.00 stress
    162 root      20   0    8864   5292   3244 R   0.0   0.6   0:00.01 top
```

signs of cgroup enforcement while memory intensive task running

```
root@fd0cc406de60:/# stress --vm 1 --vm-bytes 150M --timeout 60
stress: info: [171] dispatching hogs: 0 cpu, 0 io, 1 vm, 0 hdd
stress: FAIL: [171] (425) <-- worker 172 got signal 9
stress: WARN: [171] (427) now reaping child worker processes
stress: FAIL: [171] (461) failed run completed in 0s
root@fd0cc406de60:/# stress: info: [169] successful run completed in 120s
```

Host stats while cpu intensive task running

```
CONTAINER ID   NAME        CPU %     MEM USAGE / LIMIT   MEM %     NET I/O           BLOCK I/O         PIDS
fd0cc406de60   load-test   50.06%    6.383MiB / 100MiB   6.38%     34.8MB / 60.2kB   82.8MB / 92.1MB   3
```

Conclusion

Under heavy load, the container was strictly capped: CPU usage peaked at **50%** despite stress trying to use more, and memory allocation failed instantly when exceeding **100MiB**, triggering the **OOM killer**. docker stats and top confirmed real-time enforcement no overconsumption occurred. This proves **cgroups actively throttle CPU and kill memory-hungry processes**, ensuring container isolation and host protection.

7. Filesystem Layer Analysis

Approach

Started with a container and added some files inside its /tmp directory. On the host, I used Docker's inspect tool to locate the container's writable layer, where I could see those files

stored. After stopping and removing the container, the writable layer was completely deleted, while the original image layers remained untouched for reuse.

1. **Started a container**
   Ran **docker run -it --name fs-test ubuntu bash** to get an interactive session.



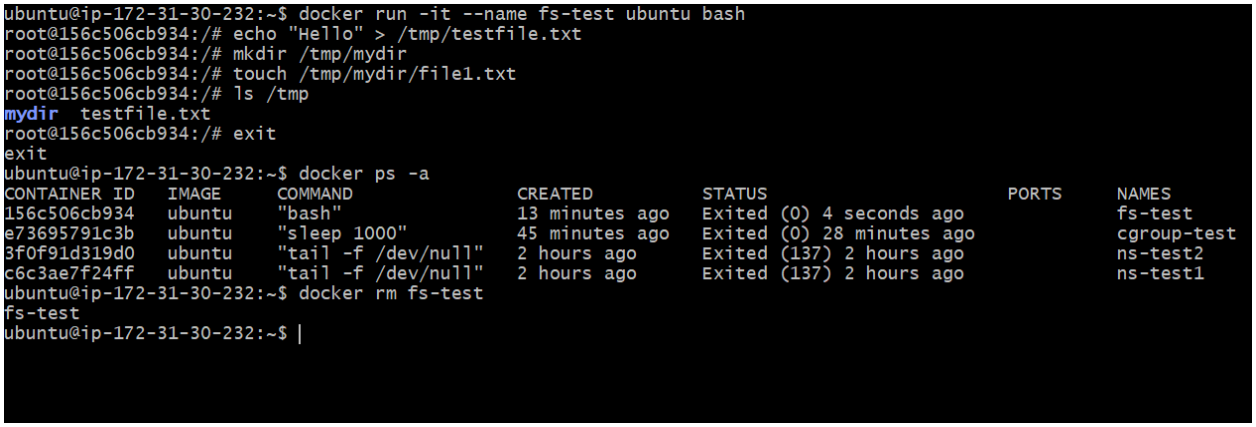
2. **Created test files**
   Inside, ran echo "Hello" > /tmp/testfile.txt, mkdir /tmp/mydir, and touch /tmp/mydir/file1.txt to generate data.

3. **Located container storage**
   On the host, used docker inspect --format '{{.GraphDriver.Data.UpperDir}}' fs-test to find the **writable layer path** (e.g., /var/lib/docker/overlay2/.../diff).



4. **Checked files on host**
   Ran sudo ls <UpperDir>/tmp — saw testfile.txt, mydir, and file1.txt persisted in the **writable layer**.

5. **Exited and removed container**
   Ran exit then docker rm fs-test.



6. **Verified cleanup**

Confirmed the writable layer directory was **deleted** from **/var/lib/docker/overlay2/.**

```
ubuntu@ip-172-31-30-232:/$ docker ps -a
CONTAINER ID   IMAGE     COMMAND               CREATED          STATUS                      PORTS     NAMES
156c506cb934   ubuntu    "bash"                11 minutes ago   Up 11 minutes                         fs-test
e73695791c3b   ubuntu    "sleep 1000"          44 minutes ago   Exited (0) 27 minutes ago             cgroup-test
3f0f91d319d0   ubuntu    "tail -f /dev/null"   2 hours ago      Exited (137) 2 hours ago              ns-test2
c6c3ae7f24ff   ubuntu    "tail -f /dev/null"   2 hours ago      Exited (137) 2 hours ago              ns-test1
ubuntu@ip-172-31-30-232:/$ docker inspect --format '{{.GraphDriver.Data.UpperDir}}' fs-test
/var/lib/docker/overlay2/675e6f1a21520f65697d93eb5347de012e6c85a92f2e999e3c11ec71869482e2/diff
ubuntu@ip-172-31-30-232:/$ sudo ls /var/lib/docker/overlay2/675e6f1a21520f65697d93eb5347de012e6c85a92f2e999e3c11ec71869482e2/diff/tmp
mydir  testfile.txt
ubuntu@ip-172-31-30-232:/$ sudo ls /var/lib/docker/overlay2/675e6f1a21520f65697d93eb5347de012e6c85a92f2e999e3c11ec71869482e2/diff/tmp
ls: cannot access '/var/lib/docker/overlay2/675e6f1a21520f65697d93eb5347de012e6c85a92f2e999e3c11ec71869482e2/diff/tmp': No such file or directory
ubuntu@ip-172-31-30-232:/$
```

Conclusion

Docker uses **OverlayFS** with **read-only image layers** (shared) and a **writable container layer**. Files created inside the container are stored only in the writable layer. When the container is removed, this writable layer is deleted, ensuring **data cleanup**, while image layers stay intact for reuse.

8. Process Creation Flow Examination

Approach

I started a container named **pstree-test running sleep 1000**. Using **docker inspect,** I found its **host PID (9981)**. Then, I ran sudo **pstree -ps 9981** on the host to trace the full process ancestry.

```
ubuntu@ip-172-31-30-232:/$ docker ps -a
CONTAINER ID   IMAGE     COMMAND               CREATED          STATUS                      PORTS     NAMES
9640f67f3a2a   ubuntu    "sleep 1000"          7 minutes ago    Up 7 minutes                          pstree-test
e73695791c3b   ubuntu    "sleep 1000"          58 minutes ago   Exited (0) 42 minutes ago             cgroup-test
3f0f91d319d0   ubuntu    "tail -f /dev/null"   2 hours ago      Exited (137) 2 hours ago              ns-test2
c6c3ae7f24ff   ubuntu    "tail -f /dev/null"   3 hours ago      Exited (137) 2 hours ago              ns-test1
ubuntu@ip-172-31-30-232:/$ docker inspect --format '{{.State.Pid}}' pstree-test
9981
ubuntu@ip-172-31-30-232:/$ sudo pstree -ps 9981
systemd(1)───containerd-shim(9959)───sleep(9981)
ubuntu@ip-172-31-30-232:/$
```

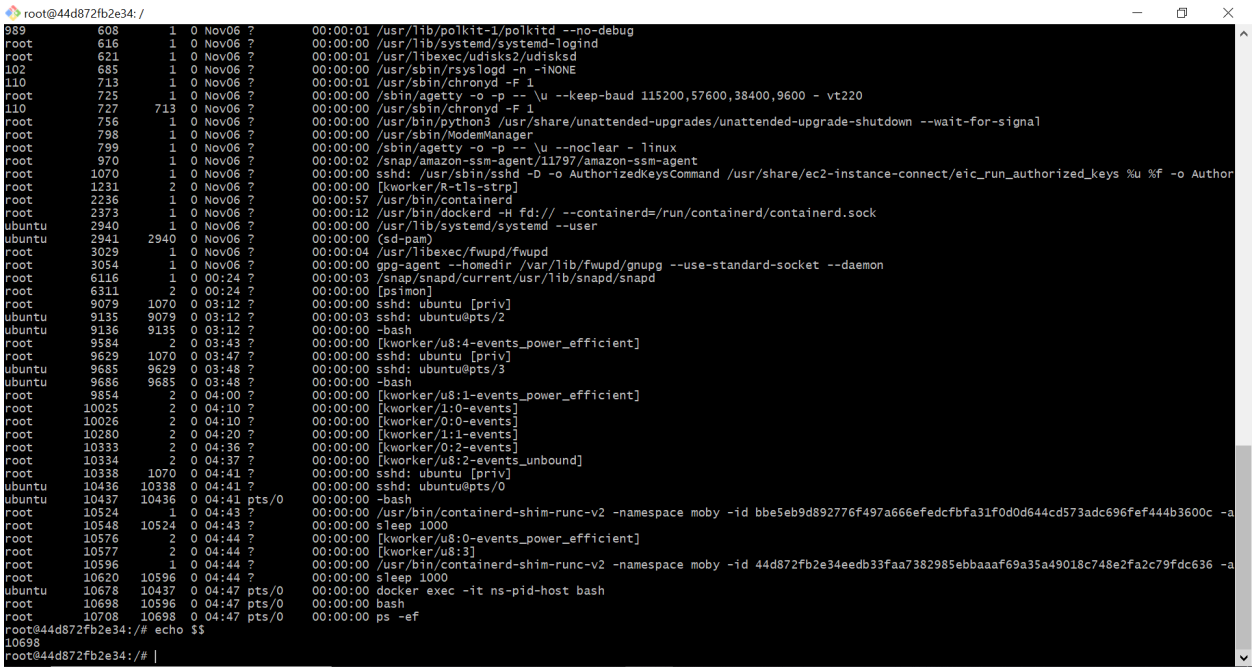
Conclusion

The process tree clearly shows **containerd-shim** as the direct parent of the container process (sleep). This confirms that **containerd-shim** is the persistent runtime process managing the container, launched by systemd, and it directly spawns the container's main process (PID 1 inside). There's no visible dockerd or runc in the live tree because runc exits after setup, and containerd-shim takes over. This proves **containers are lightweight Linux processes** under **containerd-shim**, fully integrated into the host's process hierarchy.

9. Comparative Namespace Experiment

Approach

I launched two containers, one with **default isolation** (ns-default) and another with **host PID namespace sharing** (ns-pid-host) using --pid=host. Inside the default container, **ps -ef** showed only its own sleep process (PID 1). In the --pid=host container, **ps -ef** revealed **all host processes**, including systemd, containerd-shim, and other containers. This proved the host PID namespace was **fully shared**.



```
ubuntu@ip-172-31-30-232:~$ docker run -d --name ns-default ubuntu sleep 1000
bbe5eb9d892776f497a666efedcfbfa31f0d0d644cd573adc696fef444b3600c
ubuntu@ip-172-31-30-232:~$ docker run -d --name ns-pid-host --pid=host ubuntu sleep 1000
44d872fb2e34eedb33faa7382985ebbaaaf69a35a49018c748e2fa2c79fdc636
ubuntu@ip-172-31-30-232:~$ docker exec -it ns-default bash
root@bbe5eb9d8927:/# ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 04:43 ?        00:00:00 sleep 1000
root           7       0  0 04:46 pts/0    00:00:00 bash
root          15       7  0 04:46 pts/0    00:00:00 ps -ef
root@bbe5eb9d8927:/# ech $$
bash: ech: command not found
root@bbe5eb9d8927:/# echo $$
7
root@bbe5eb9d8927:/# exit
exit
ubuntu@ip-172-31-30-232:~$ docker exec -it ns-pid-host bash
root@44d872fb2e34:/# ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 Nov06 ?        00:00:05 /sbin/init
root           2       0  0 Nov06 ?        00:00:00 [kthreadd]
root           3       2  0 Nov06 ?        00:00:00 [pool_workqueue_release]
root           4       2  0 Nov06 ?        00:00:00 [kworker/R-rcu_gp]
root           5       2  0 Nov06 ?        00:00:00 [kworker/R-sync_wq]
root           6       2  0 Nov06 ?        00:00:00 [kworker/R-kvfree_rcu_reclaim]
root           7       2  0 Nov06 ?        00:00:00 [kworker/R-slub_flushwq]
root           8       2  0 Nov06 ?        00:00:00 [kworker/R-netns]
root          11       2  0 Nov06 ?        00:00:00 [kworker/0:0H-events_highpri]
root          13       2  0 Nov06 ?        00:00:00 [kworker/R-mm_percpu_wq]
root          14       2  0 Nov06 ?        00:00:00 [rcu_tasks_rude_kthread]
root          15       2  0 Nov06 ?        00:00:00 [rcu_tasks_trace_kthread]
root          16       2  0 Nov06 ?        00:00:00 [ksoftirqd/0]
root          17       2  0 Nov06 ?        00:00:01 [rcu_sched]
root          18       2  0 Nov06 ?        00:00:00 [rcu_exp_par_gp_kthread_worker/0]
root          19       2  0 Nov06 ?        00:00:00 [rcu_exp_gp_kthread_worker]
root          20       2  0 Nov06 ?        00:00:00 [migration/0]
root          21       2  0 Nov06 ?        00:00:00 [idle_inject/0]
root          22       2  0 Nov06 ?        00:00:00 [cpuhp/0]
root          23       2  0 Nov06 ?        00:00:00 [cpuhp/1]
root          24       2  0 Nov06 ?        00:00:00 [idle_inject/1]
root          25       2  0 Nov06 ?        00:00:00 [migration/1]
root          26       2  0 Nov06 ?        00:00:00 [ksoftirqd/1]
root          28       2  0 Nov06 ?        00:00:00 [kworker/1:0H-events_highpri]
root          29       2  0 Nov06 ?        00:00:00 [kdevtmpfs]
root          30       2  0 Nov06 ?        00:00:00 [kworker/R-inet_frag_wq]
root          31       2  0 Nov06 ?        00:00:00 [kauditd]
root          32       2  0 Nov06 ?        00:00:00 [khungtaskd]
root          34       2  0 Nov06 ?        00:00:00 [oom_reaper]
root          36       2  0 Nov06 ?        00:00:00 [kworker/R-writeback]
```

Conclusion

By default, Docker creates **new PID namespaces** for isolation containers see only their own processes. With --pid=host, Docker uses **setns()** to **join the host's PID namespace** instead of creating a new one. This makes **all host processes visible and manageable** from inside the container, breaking isolation.